C语言——面向过程编程

单元三、项目重构

第一次课 函数

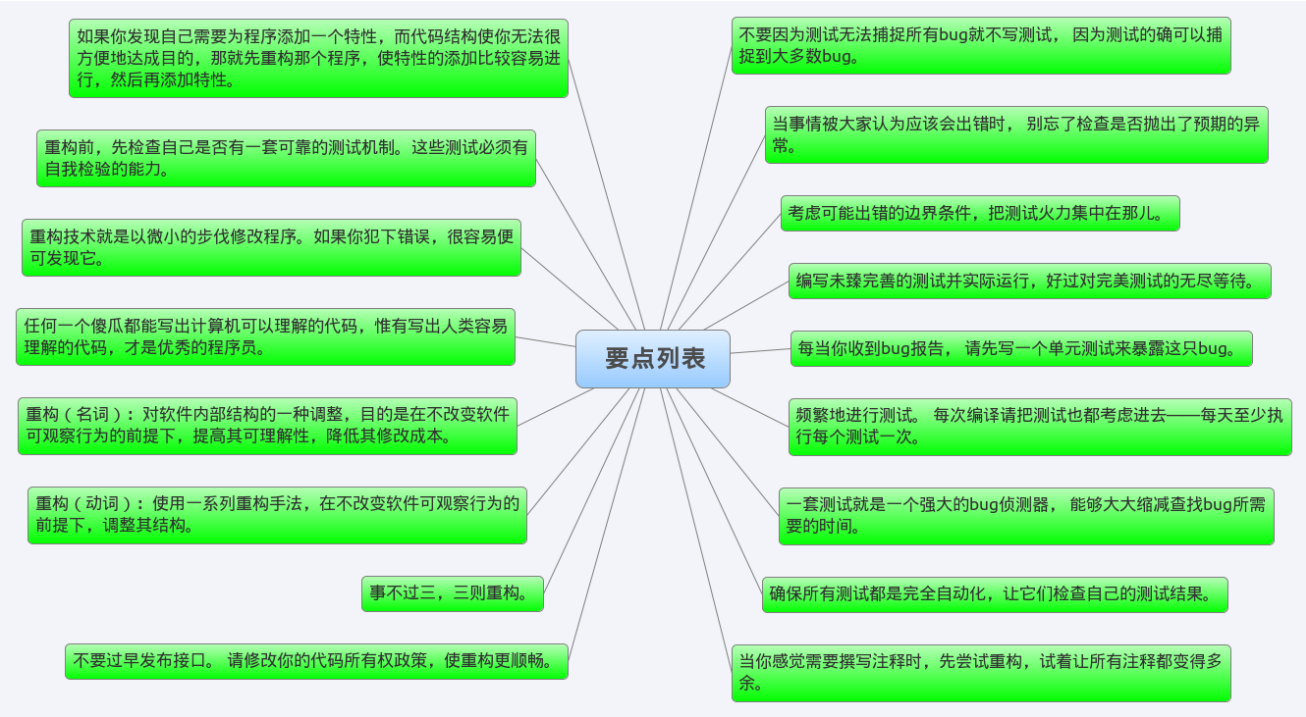
一. 《重构》思想

重构（名词）：对软件**内部结构的**一种**调整**，目的是**在不改变软件可观察行为的前提下**，**提高**其可理解性，降低其修改成本。

重构（动词）：使用一系列重构手法，在不改变软件可观察行为的前提下，调整其结构。

重构可以使软件更容易地被修改和被理解。通过不断地改进软件设计以达到简单设计的目标，减少由于设计与业务的不匹配带来的架构与设计腐化。

掌握了重构的手法和心法，会让重构变得更加简单安全高效可控，从而真正的发挥出其巨大的威力，让我们的软件永葆青春。



《重构，改善既有代码的设计》读书笔记

二. 函数的基本概念

A： 函数的意义

通过printf()函数、rand()函数以及easyX游戏框架提供的函数来理解函数是什么：

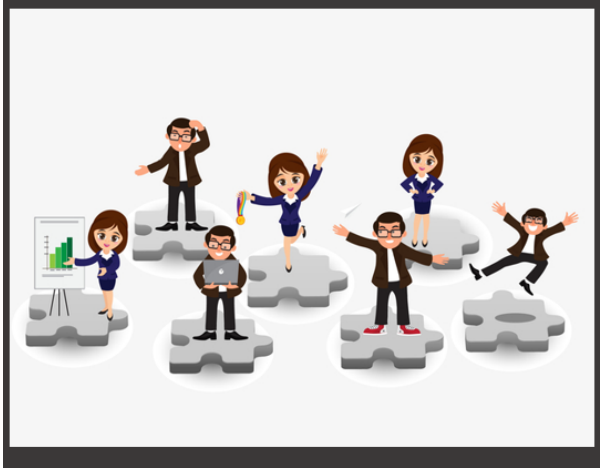
①：函数不同于数学定义的函数，它是一个具体代码逻辑功能的描述与使用

②：在程序中；**函数**就是**封装**了一段**流程结构**的代码，是一种重构的技术手段。

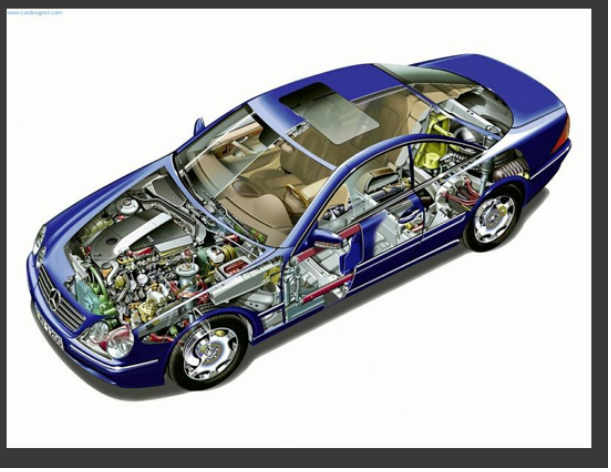
③：重构后的代码结构可增强代码的可维护性、可阅读性、可复用性等。

④：函数更利于结构性、模块化的程序设计与分工协作。

每一个团队成员的具体工作都可以是一个函数



每一个汽车零部件可以是一个函数



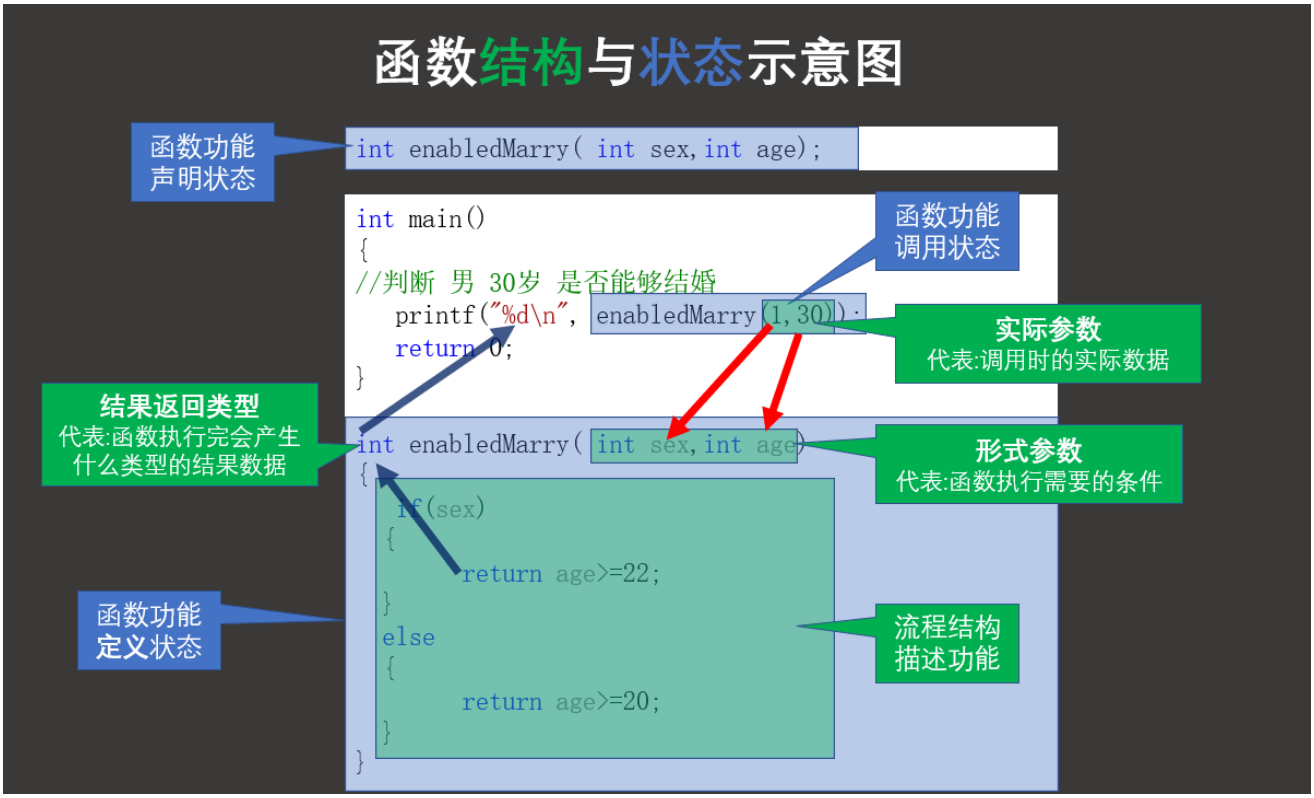
B：函数的基本结构

```
//编写函数：判断一个人是否可以结婚
//函数的定义状态：描述函数功能的实现过程，就是一段流程结构的封装
int enabledMarry( int sex,int age)
{
    if(sex)//逻辑真非0 代表男
    {
```

```
        return age>=22;
    }
    else
    {
        return age>=20;
    }
}
int main()
{
    //判断 男 30岁 是否能够结婚
    printf("%d\n", enabledMarry(1,30));//函数的调用状态
    return 0;
}
```

如果：enabledMarry自定义函数在main函数后面；即调用在前，定义在后。则需要在调用前做一下函数的**前置声明**。

```
int enabledMarry( int sex,int age);//函数声明状态：  写在调用之前，只写函数头，不写实现过程

int main()
{
    //判断 男 30岁 是否能够结婚
    printf("%d\n", enabledMarry(1,30));//函数的调用状态
    return 0;
}

//函数的定义状态：描述函数功能的实现过程，就是一段流程结构的封装
int enabledMarry( int sex,int age)
{
    if(sex)//逻辑真非0 代表男
    {
        return age>=22;
    }
    else
    {
        return age>=20;
    }
}
```



【关于：return 返回】

```
void fun(void)//void 代表空 放在返回值位置上，说明此函数执行完不返回任何值；放在参数位置上则显示说明无形式参数
{

    return; //返回值为void 最后一句可以不写return
}
```

```
int fun(void)
{

    return 3;//return 表达式 的发挥结果要么和返回类型一致，要么能自动类型转换之。
}
```

```
void fun()
{
    int a=3;
    return ;//return执行会终止函数的这次调用，后面的代码则“不可到达”
    a++;//此处代码不会被执行，甚至产生编译错误。
}
```

如果需要中通停止函数执行，则需要用if条件语句

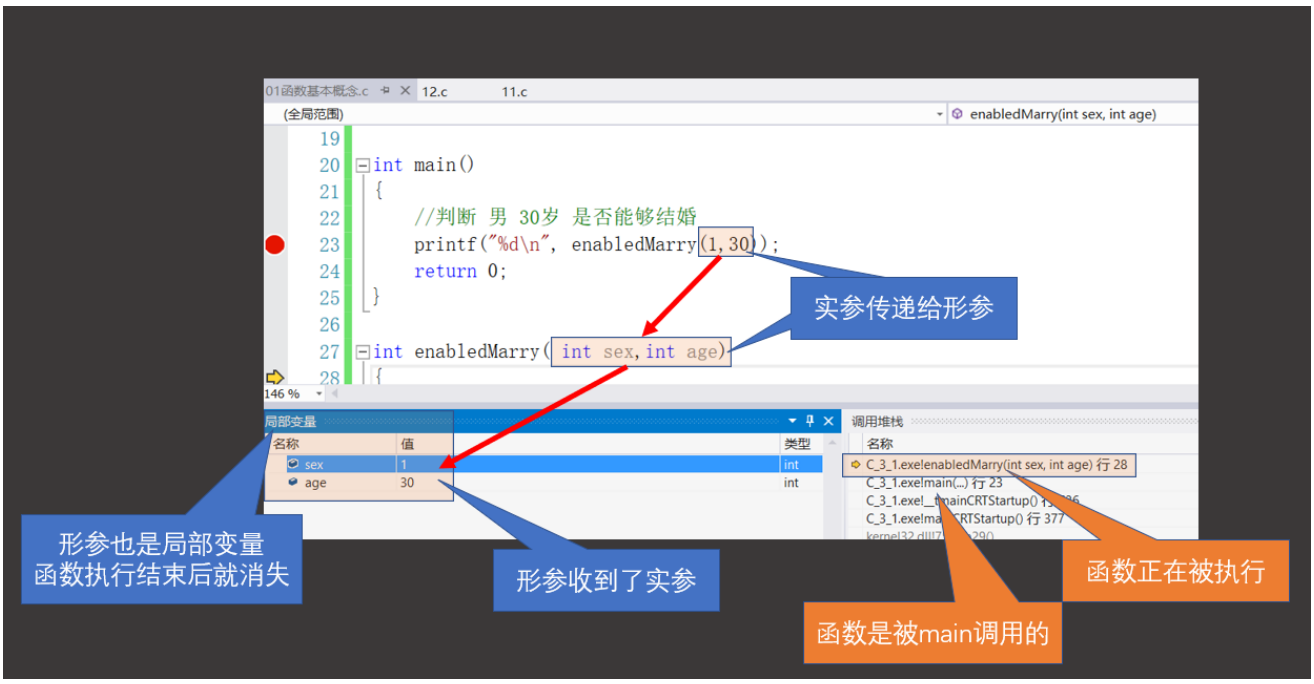
```
void fun()
{
    int a=3;
    if(a<3)
    {
        return ;
    }
    a++;
}
```

三、函数的执行原理

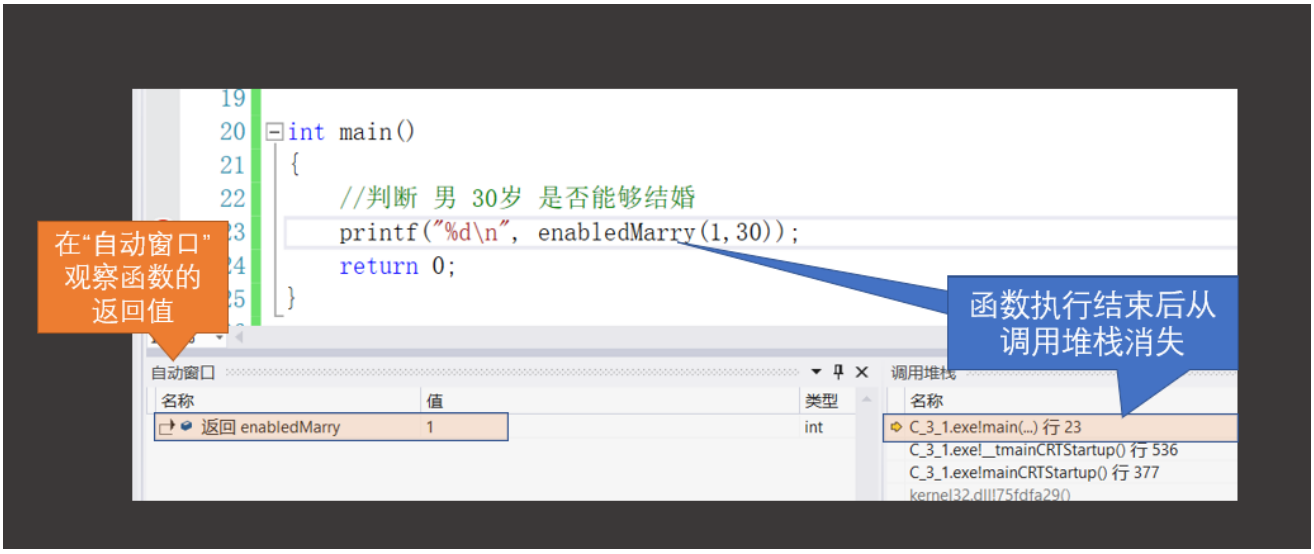
① 函数只有被调用才执行，不调用不执行。

调用前的 “调用堆栈”

点击 进入调用执行过程中



逐行执行到 `return age>=22;` 时，将**结束**函数的**执行**，并**返回**一个计算**结果**

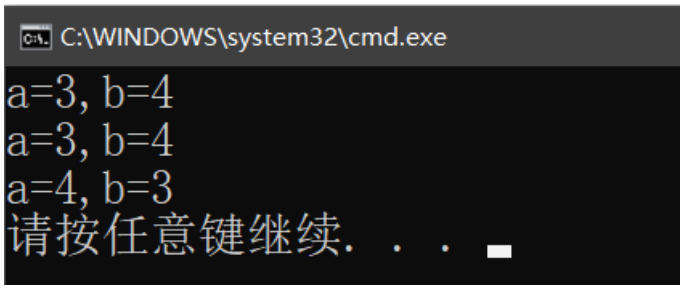


② 函数之间**数据值传递**为【复制方式】

阅读如下代码：

```
/*交换参数变量a b的值*/
void exchange(int a,int b)
{
    int t;
    t=a;a=b;b=t;
}
/*交换参数指针变量p、q所指向空间的值*/
void swap(int*p,int *q)
{
    int t;
    t=*p;*p=*q;*q=t;
}

int main(void)
{
    int a=3,b=4;
    printf("a=%d,b=%d\n",a,b);
    exchange(a,b);
    printf("a=%d,b=%d\n",a,b);
    swap(&a,&b);
    printf("a=%d,b=%d\n",a,b);
    return 0;
}
```

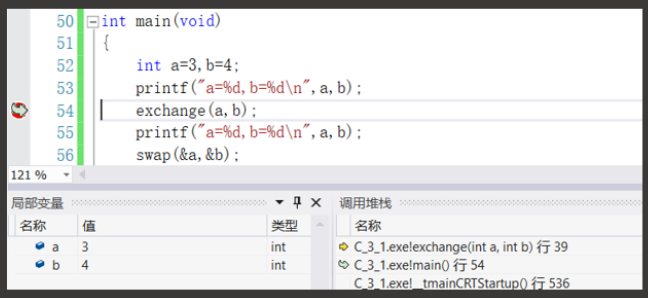
执行结果



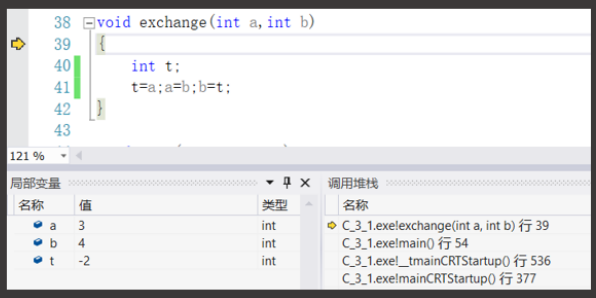
根据结果可知：exchange()函数并没有改变main()函数中a,b变量的值。但是swap()函数做到了。

分别调试两个函数的调用执行。

观察exchange函数的执行。

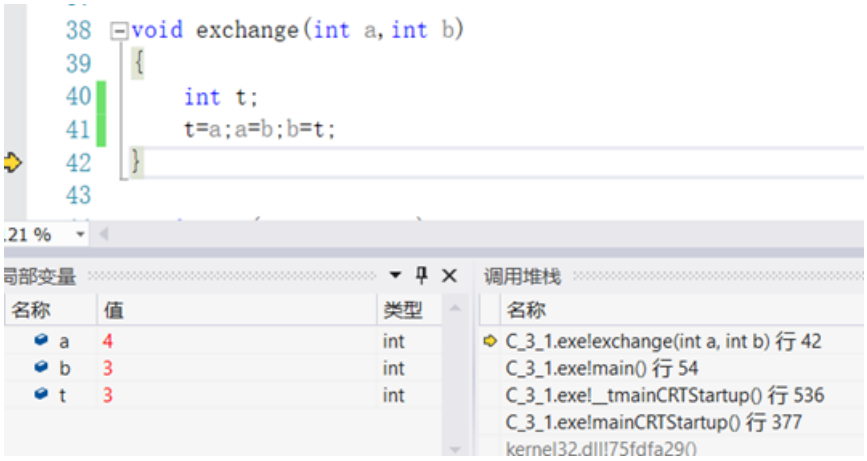


main函数的a，b局部变量



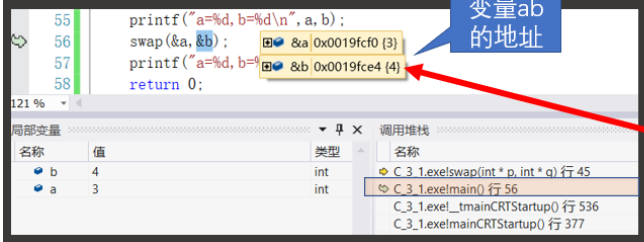
exchange函数的a，b，t局部变量

两个函数同名的局部变量a，b的值分别是3，4。exchange函数的ab值是由main中的a,b复制过去的。因此exchange改变ab变量的值不会影响到main函数的ab变量值。

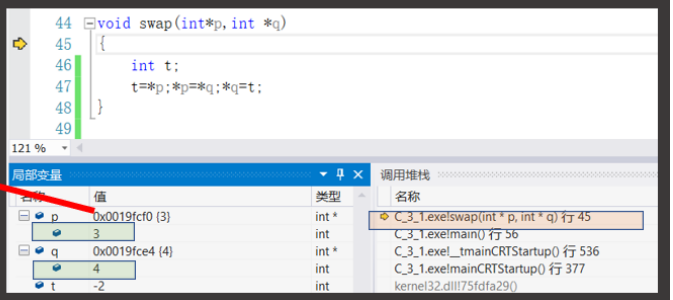


exchange函数只改变了自己参数变量ab的值

再观察swap函数的调用执行



main函数的a，b局部变量的地址



swap函数的p，q 指向了main函数ab的空间
并可通过地址查看两个变量的值：3，4

```
44  void swap(int*p, int *q)
45  {
46      int t;
47      t=*p;*p=*q;*q=t;
48  }
49
```

121 %

局部变量

名称	值	类型
p	0x0019fcf0 {4}	int *
4		int
q	0x0019fce4 {3}	int *
3		int
t	3	int

调用堆栈

名称
C_3_1.exe!swap(int * p, int * q) 行 48
C_3_1.exe!main() 行 56
C_3_1.exe!__tmainCRTStartup() 行 536
C_3_1.exe!mainCRTStartup() 行 377
kernel32.dll!75fdfa29()

swap函数执行过程中：利用内存唯一的地址达到对main函数局部变量ab值的改变。此案例中main函数调用swap函数时的实参时两个指针地址。

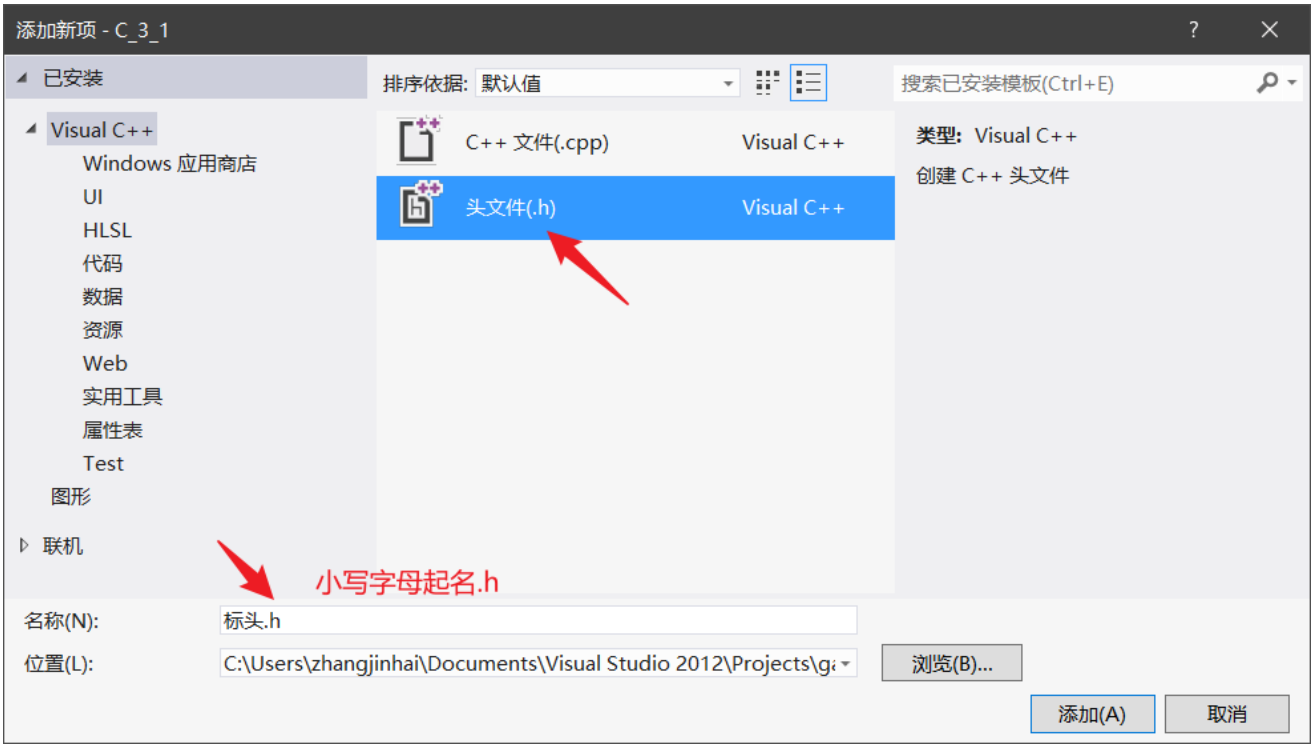
再尝试调试下面的代码：

```c
int add_all(int a[],int len )
{
    int sum=0;
    int i;
    for(i=0;i<len;i++)
    {
        sum+=a[i];
    }
    return sum;
}

int main(void)
{
    int arr[]={1,2,3,4,5,6};
    int h=add_all(arr,sizeof(arr)/sizeof(arr[0]));
    printf("%d\n",h);
    return 0;
}
```

四、自定义头文件

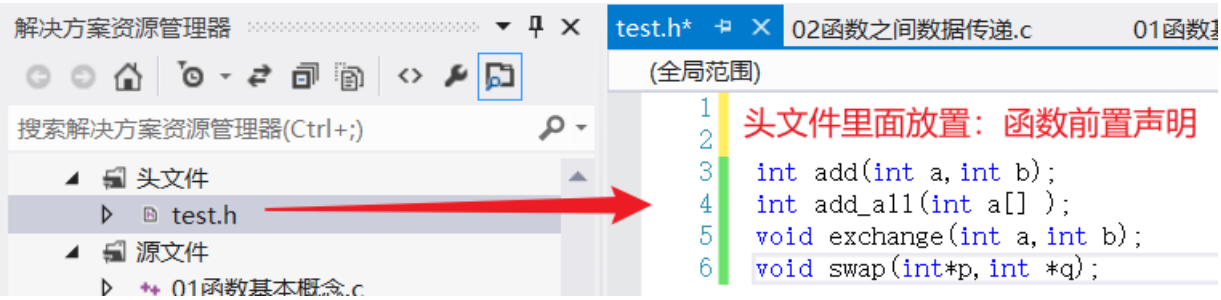
① 创建头文件

②



文件名最好全部小写字母，扩展名为.h

③ 把前置函数前置声明的代码剪切到头文件



④ 源代码.c文件则需要#include “自定义.h”的头文件

完整代码：

test.h 把自定义函数的声明放到头文件

```
int add(int a,int b);
int add_all(int a[],int len )
void exchange(int a,int b);
void swap(int*p,int *q);
```

test.c 把函数的定义（或称功能实现）放在一个 .c文件 通常声明的头文件与定义的源代码文件名称相同。

```
void exchange(int a,int b)
{
    int t;
    t=a;a=b;b=t;
}

void swap(int*p,int *q)
{
    int t;
    t=*p;*p=*q;*q=t;
}
int add(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}

int add_all(int a[],int len )
{
    int sum=0;
    int i;
    for(i=0;i<len;i++)
    {
        sum+=a[i];
    }
    return sum;
}
```

xxxx.c 调用test.c中的自定义函数

```
#include <stdio.h>
#include "test.h"//包含自定义函数的声明头文件后即可使用相关函数。
int main(void)
{
    int arr[]={1,2,3,4,5,6};
    int h;
    int a=3,b=4;
    printf("a=%d,b=%d\n",a,b);

    exchange(a,b);
    printf("a=%d,b=%d\n",a,b);

    swap(&a,&b);
    printf("a=%d,b=%d\n",a,b);

    h=add_all(arr,sizeof(arr)/sizeof(arr[0]));
    printf("%d\n",h);

    return 0;
}
```

五、函数指针

① 函数名的本质

函数名代表了函数代码在内存中的地址，像数组名一样；也是一个地址常量。

(全局范围)

```
#include <stdio.h>

int add_int(int a, int b);


int main    ()
{

    printf("%d\n", add_int);
    printf("%p\n", add_int);
    return 0;
}

int add_int(int a, int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

打印函数名：会看到其内存地址

C:\WINDOWS\system32\cmd.exe

```
5771554
00581122
请按任意键继续. . .
```

② 对函数名取地址

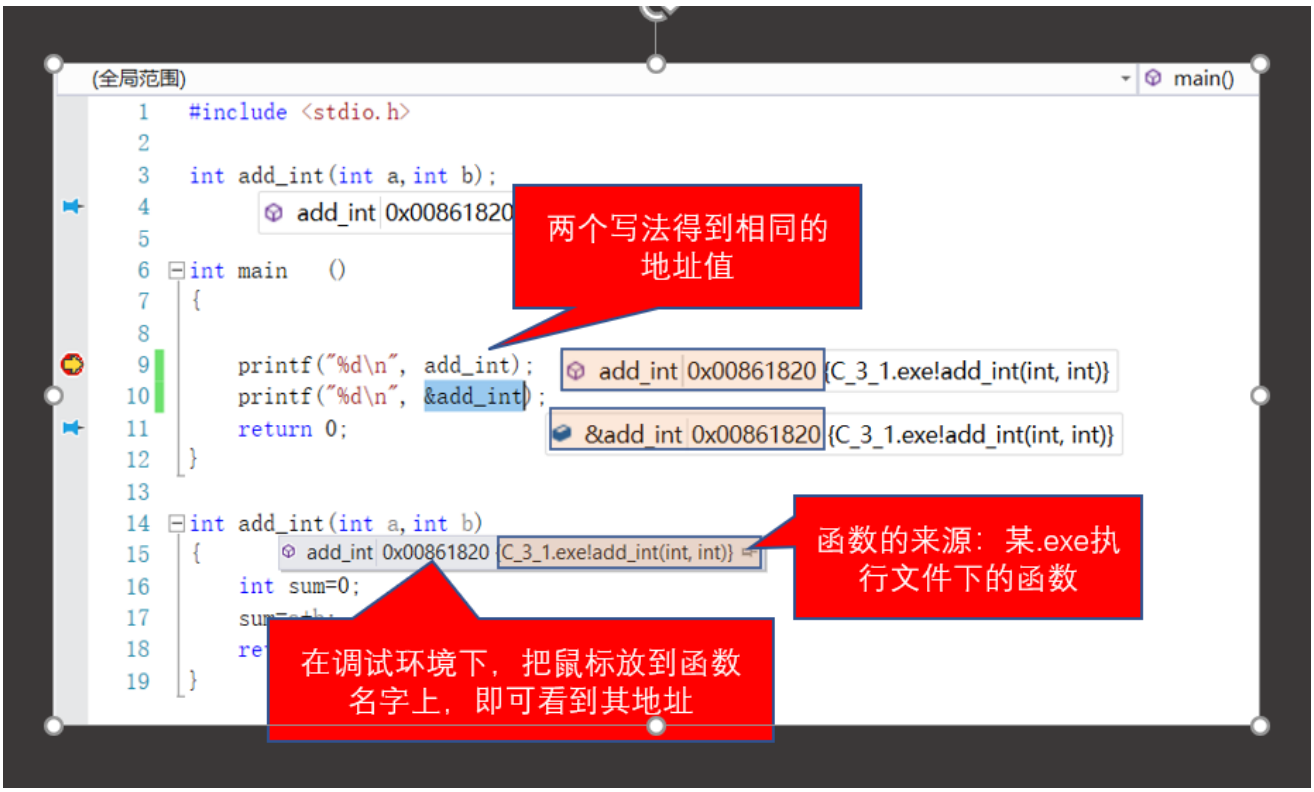
```
#include <stdio.h>

int add_int(int a, int b);


int main    ()
{

    printf("%d\n", add_int);
    printf("%d\n", &add_int);
    return 0;
}

int add_int(int a, int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

对函数名取地址得到相同的地址

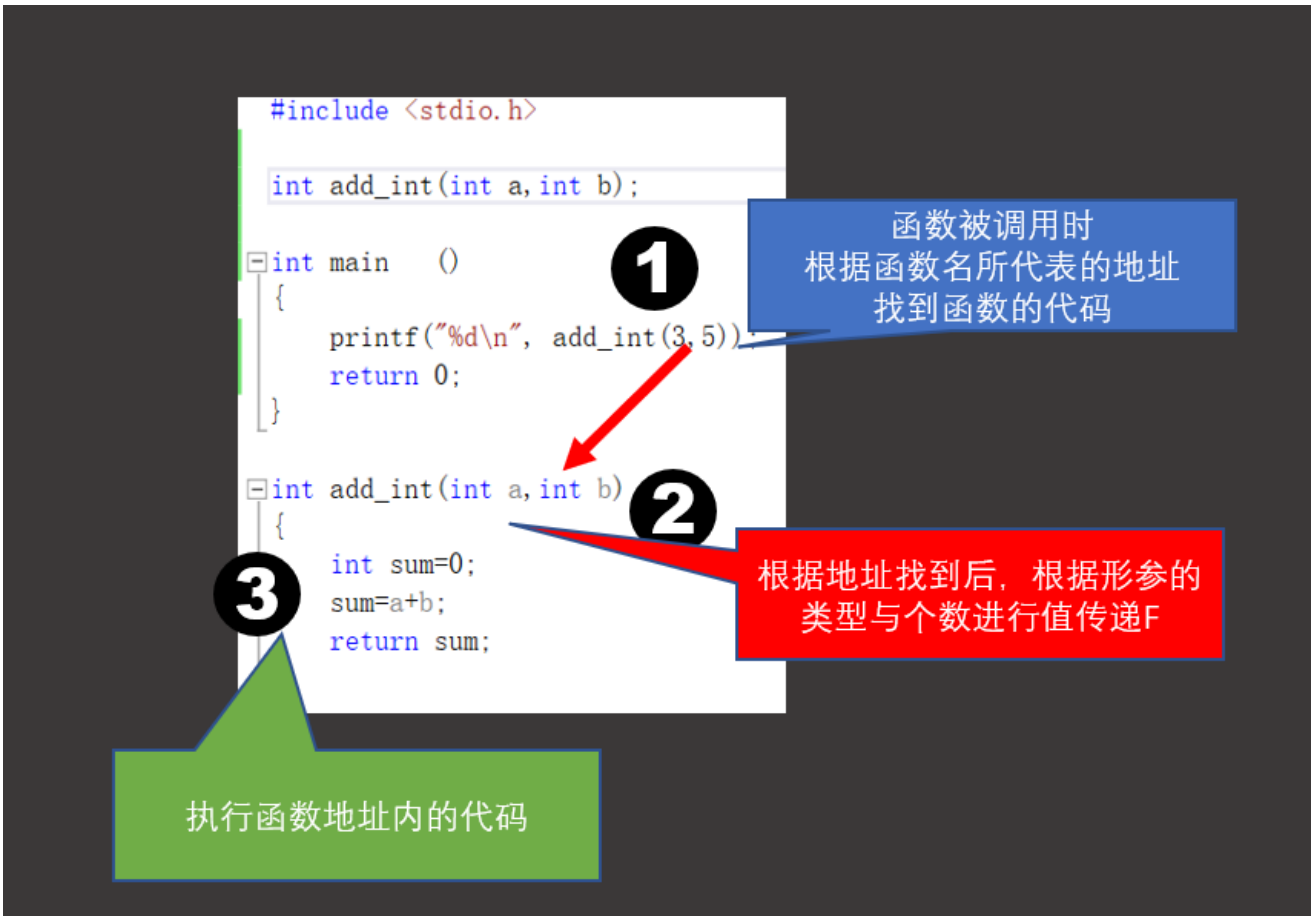
C:\WINDOWS\system32\cmd.exe

```
12521762
12521762
请按任意键继续. . .
```

③通过调试器查看



④函数被调用时，能够执行的本质：**地址+实际参数**



⑤ 函数指针的声明

既然函数名是地址，就可以用指针变量来保存。根据之前指针的使用经验：指针变量必须强调是什么类型的指针。因此，函数的指针定义时必须把函数的特征表述清楚。就如同一维数组指针与二维数组指针的定义一样：回忆如下代码

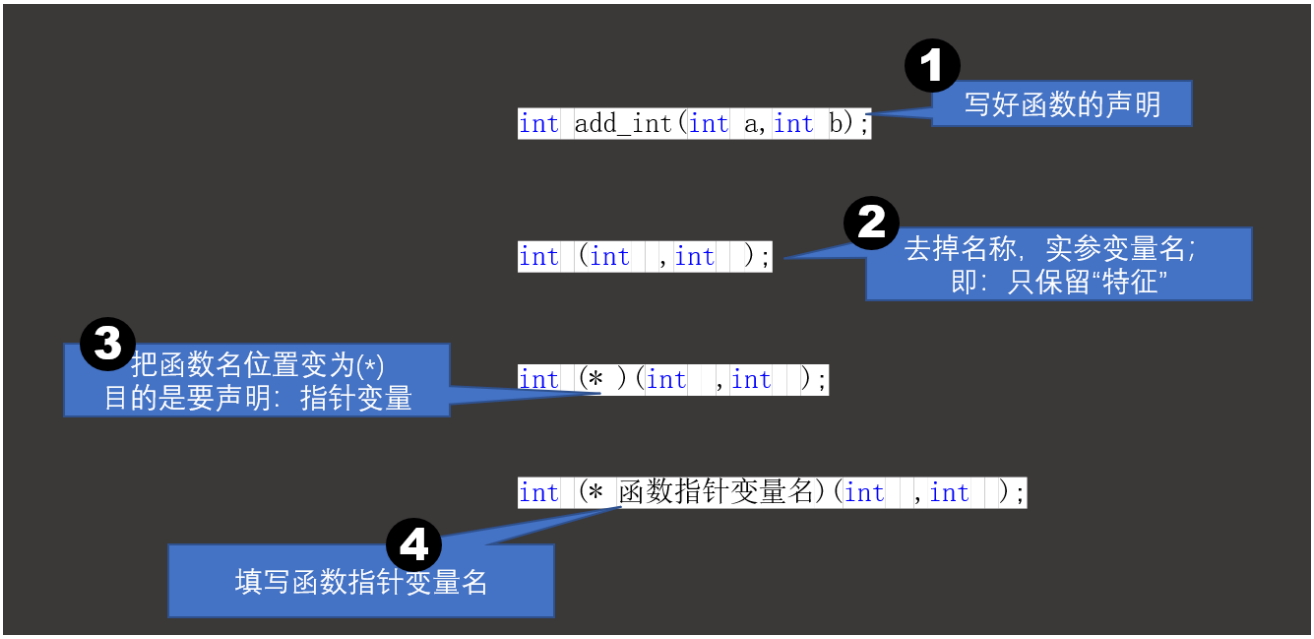
```
//一维数组与指针
int a[5];// 一维数组的特征 int  [5]
int (*p)[5];//一维数组指针保留特征 int [5] 然后在 名字位置上写成：(* 指针变量名)
//二维数组与指针
int arr[5][3];// 二维数组的特征 int  [5][3]
int (*p)[5][3];//二维数组指针保留特征 int [5][3] 然后在 名字位置上写成：(* 指针变量名)
```

基于以上模式，函数指针的声明方式：函数特征+(*指针变量名)

其中函数特征包括：返回值类型， 参数个数及每个参数的类型不理会实参变量名具体是什么。如：

```
#include <stdio.h>

int add_int(int a,int b);

int main   ()
{
    int  (* pfun)(int  ,int  );//声明一个能指向add_int的函数指针
    //为函数指针赋值，使其能指向函数代码，并能执行所指向的函数代码。
    pfun=add_int;//方式1：把函数名赋给函数指针变量
    pfun=&add_int;//方式2：把函数名先取地址，再把计算的地址赋给函数指针变量
    return 0;
}

int add_int(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```

【注意：】定义函数指针变量的过程



⑥ 利用函数指针执行函数代码。

方式1：(*函数指针) (实际参数) ← 推荐

方式2： 函数指针 (实际参数)

```
#include <stdio.h>
```

```c
int add_int(int a,int b);

int main   ()
{
    int  (* pfun)(int  ,int  );//声明一个能指向add_int的函数指针
    //为函数指针赋值，使其能指向函数代码，并能执行所指向的函数代码。
    pfun=add_int;//方式1：把函数名赋给函数指针变量
    pfun=&add_int;//方式2：把函数名先取地址，再把计算的地址赋给函数指针变量
    //利用函数指针变量 pfun执行其所指向的代码。
    printf("%d\n", (*pfun)(10,20));//显式的使用，让人一目了然它是一个函数指针
    printf("%d\n",pfun(11,22));//隐式的使用，让人看不出它是否是函数指针，字面上与下行代码无异
    printf("%d\n",add_int(111,222));//通过函数名调用函数
    return 0;
}

int add_int(int a,int b)
{
    int sum=0;
    sum=a+b;
    return sum;
}
```